Operations & Examination Management Portal

Software Requirements Specification (SRS)

Document attribute	Details
Document type	Standalone Software Requirements Specification
Version	1.0
Status	Draft for Business and Technical Review
Prepared for	Operations & Examination Management Portal
Related reference	Functional Requirements Document (FRD) – maintained separately
Purpose	Technical, architecture, automation, security, testing and deployment specification

This SRS is intentionally separate from the Functional Requirements Document. The FRD remains the authoritative source for module-wise fields, detailed workflows, permissions and business process rules. This document specifies how the portal should be engineered, tested, secured, automated and deployed.

1. Document Control

Version	Date	Description	Owner
1.0	04 September 2026	Initial standalone SRS	Product / Technology Team

2. Purpose

The purpose of this SRS is to define the technical and non-functional requirements for building the Operations & Examination Management Portal as a secure, role-based web application. It covers system architecture, data design, APIs, automation, integrations, security, testing, CI/CD and operational readiness.

3. Scope

3.1 In Scope

- A centralized web portal for Admin, Project Manager, Operations Executive, Auditor / IT Team Member, Accounts / Finance, Centre User and Management users.
- Controlled workflows for projects, centres, eligibility, booking, consent, documents, audits, technical scans, allocation, commercial records, invoicing, payments, readiness, incidents, tickets, communication, MIS and audit trails.
- Role-based access control at module, client, project, centre and financial-data scope.
- Document upload, private storage, metadata, expiry, versioning and approval controls.
- Dashboard, reporting, notification, reminder and escalation capabilities.
- API/event readiness for WhatsApp, email-to-ticket, telephony and other future integrations.

3.2 Out of Scope

- Replacement of external accounting/ERP systems unless separately approved.
- Development of a proprietary WhatsApp, telephony or email infrastructure; the portal will integrate with selected providers.
- Hardware procurement, network installation or physical technical audit execution.
- Advanced predictive analytics unless included in a separately approved future release.

4. System Overview

The solution shall be a browser-based, responsive portal with a secure Centre/Venue user experience and internal operations dashboards. The application shall operate as the system of record for project, centre, booking, workflow, document, audit, allocation, commercial, invoice, payment and accountability data.

Component	Responsibility
Internal Portal	Operations, audit, finance, administration and management workflows.
Centre Portal	Centre consent, data submission, document upload, readiness support and invoice submission.
Core API	Authorization, validation, workflow transitions, calculations, audit recording and data access.
Operational Database	Master data, transaction records, statuses, versions, approvals and history.
Automation Layer	Reminders, notifications, escalations, scheduled checks and external connector orchestration.
File Repository	Private storage of documents, images, quotations, agreements, invoices and audit evidence.
Reporting Layer	Operational, compliance, financial and executive dashboards.

5. User Classes

User class	Primary system access
Admin	User/role administration, scopes, configuration, master control, emergency controls and audit logs.
Project Manager	Project lifecycle, approvals, oversight, escalations and closure.
Operations Executive	Booking, centre coordination, documents, allocation, calling, readiness and incidents.
Auditor / IT Team Member	Assigned audit, evidence, technical scans, findings and submission.
Accounts / Finance	Commercial approvals, invoices, payments, reconciliation and finance closure.
Centre User	Own centre invitation, consent, submissions, documents, audit support, readiness and invoices.
Management / CEO	Scope-controlled dashboard, KPI and exception visibility.

6. Functional Boundary

The detailed module specification, fields, permissions, state transitions and acceptance rules are defined in the separate FRD. This SRS requires the implementation to support the following module boundaries and cross-module dependencies.

Module group	Implementation responsibility
Platform foundation	Identity, RBAC, scope authorization, configuration, templates, audit log and notifications.
Core operations	Projects, Centre Master, eligibility, booking, consent and documents.
Compliance	Audits, technical scans, exceptions, rework and re-audit.
Execution	Allocation, calling, readiness, mock, incidents, tickets, inventory and communication.
Commercial and finance	Cost sheets, versions, agreements, invoice requests, invoices, payments and closure.
Reporting	Role-filtered MIS, operational reports, finance reports, exports and management dashboards.

7. Technical Architecture

7.1 Recommended Stack

Layer	Technology recommendation	Requirement
Frontend	React, TypeScript, Vite, React Router, TanStack Query, React Hook Form, Zod, MUI/Ant Design	Responsive role-based interface, reusable forms, tables, dashboards and workflow views.
Backend	Node.js, TypeScript, NestJS or structured Express	REST APIs, domain services, workflow orchestration, authorization and integration endpoints.
Database	MongoDB Atlas or managed MongoDB	Operational records, master data, versions, workflow states and history.
Cache/Queue	Redis with BullMQ	Soft locks, delayed jobs, reminders, retries, rate limits and background work.
Object Storage	AWS S3, Azure Blob or equivalent	Private documents, photos, invoices, agreements and evidence.
Automation	n8n, self-hosted or managed	Event-driven notifications, scheduled checks and third-party integrations.
Observability	Sentry, OpenTelemetry and cloud logs/metrics	Error tracking, performance monitoring, job monitoring and alerts.
CI/CD	GitHub Actions, GitLab CI or equivalent	Build, test, scan, deploy, release and rollback pipeline.

7.2 Architecture Principles

- Use a modular monolith for the MVP. Modules shall be independently organized in code, but deployed as one primary application until scale or ownership requires extraction.
- The backend and database shall be the authoritative source of truth for business data, approvals, permissions, locks, versions and audit records.
- n8n shall orchestrate integrations and automation but shall not be the source of truth for booking, approval, finance, audit or workflow records.
- Use domain events and an outbox pattern for reliable handoff from application events to notification and integration workflows.
- Use asynchronous workers for non-blocking activities such as reminders, email delivery, file processing, imports, scan ingestion and reporting refresh.
- Use synchronous, transactional application logic for booking confirmation, critical approval transitions, financial posting and final allocation locking.

8. Data Architecture

8.1 Data Domains

Domain	Representative entities / collections
Identity and access	users, roles, permissions, scopes, sessions, invitations
Master data	clients, exams, centres, centre_contacts, centre_infrastructure, configurations
Project operations	projects, project_cities, project_team, milestones, bookings, booking_locks, consents
Eligibility	restrictions, eligibility_decisions, overrides, review_tasks
Documents	document_types, checklists, documents, document_versions, file_metadata
Audit and technical	audits, audit_checklists, findings, corrective_actions, scan_runs, devices, scan_exceptions
Allocation	allocations, allocation_versions, allocation_rows, change_requests
Commercial and finance	quotations, cost_sheets, cost_sheet_versions, agreements, invoice_requests, invoices, payments
Accountability	tasks, calls, callbacks, readiness_items, mocks, incidents, tickets, assets, logistics
Platform records	notifications, communication_logs, audit_logs, outbox_events, report_snapshots

8.2 Data Rules

- Each critical transactional record shall contain a unique system-generated identifier, status, creator, creation timestamp, update timestamp and scope references.
- Approved allocations, cost sheets, document versions, audits, consent records and commercial records shall retain historical versions.
- Cancelled, rejected, superseded and replaced records shall be retained and linked rather than silently deleted.
- Database design shall separate master, transaction, workflow, document, audit and reporting data.
- Use MongoDB transactions for multi-document operations that must succeed or fail together.
- Use references for high-volume collections such as audit logs, device scans, allocation rows and notifications.
- Store immutable snapshots in transaction records where historical accuracy is required, such as the approved centre details or commercial summary at booking time.

8.3 Required Constraints

Area	Technical control
Booking	Atomic conflict validation and unique/transactional protection for active booking combinations.
Soft locks	Dedicated lock record with expiry, ownership and release history; TTL applies only to temporary lock records.
Eligibility	Scope checks shall consider centre, client, exam, status and validity dates.
Documents	Current approved version must be identifiable while historic versions remain accessible to authorized users.
Audits	Approval must reject self-approval when reviewer equals audit submitter.
Allocation	Locked version is immutable; change creates a new version and approval cycle.
Commercial	Approved record cannot be directly overwritten; changes create a revised version.
Payments	Financial writes restricted to authorized finance roles; outstanding shall be calculated from approved invoices and verified payments.
Audit log	Critical actions and value changes must be append-only and access controlled.

9. API Requirements

- Expose versioned HTTPS REST APIs under /api/v1.
- Use request/response validation schemas and standard error response objects.
- Enforce authentication, action permission and scope authorization on every protected endpoint.
- Use server-side filtering, sorting and pagination for list APIs.
- Use idempotency keys for booking confirmation, payment posting and external-webhook retry endpoints.
- Generate OpenAPI/Swagger documentation and maintain API contracts in source control.
- Secure webhooks with signatures, timestamp validation, replay protection and IP/rate controls where applicable.
- Record audit events for sensitive reads, exports, writes, approval decisions, overrides and financial actions.

10. Automation Requirements

n8n shall be used as the automation and integration layer. The application backend shall emit reliable domain events; n8n shall consume those events, perform delivery/orchestration actions, and return delivery outcomes to the application notification log.

Automation	Trigger	Expected outcome
Consent dispatch	Consent request created	Send secure invitation, record delivery and schedule reminder.
Document reminder	Mandatory document pending	Send centre reminder, create/update task and escalate when overdue.
Expiry monitoring	Document, agreement or audit nearing expiry	Create notification and action task according to policy.
SLA escalation	Task, incident or ticket overdue	Escalate to configured roles and record escalation history.
Commercial follow-up	Quotation, agreement or invoice pending	Notify centre, operations and finance based on workflow state.
Payment follow-up	Payment due/overdue	Notify finance and permitted management users.
Daily digest	Scheduled daily execution	Send management/project health and exception summary.
Email-to-ticket	Incoming email webhook	Validate sender, create/update ticket and attach message metadata.
WhatsApp/telephony	Configured external event	Send approved communication or log call event without changing core data directly.

10.1 n8n Controls

- n8n credentials shall be encrypted and managed through a secrets manager.
- All workflows shall use environment-specific credentials and endpoints.
- n8n shall not directly modify critical database collections; it shall call authorized backend APIs.
- Every automation request shall contain an event ID or idempotency key to prevent duplicate messages/tasks.
- Retry failures using defined policy; capture success, failure, provider reference and timestamps in the portal.
- Use a staging n8n instance for testing before production activation.
- Back up n8n workflow definitions and execution configuration.

11. Frontend Requirements

- The frontend shall use React with TypeScript and a shared design system.
- The interface shall support desktop and mobile-responsive layouts for applicable workflows.
- All screens shall enforce route-level and component-level UI permission controls; backend authorization remains mandatory.
- Forms shall support required fields, conditional fields, business-validation errors, saved drafts where permitted and controlled submission.
- Workflow screens shall display current status, next action, owner, due date, approval state, version and history.
- List views shall provide server-side filtering, pagination, sort, saved filters and role-appropriate export controls.
- Dashboards shall provide KPI cards, exception lists, drill-down links and scope-aware filters.
- Accessible labels, keyboard support, visible focus states, error summaries and adequate color contrast shall be implemented.

12. Dashboard and Reporting Requirements

Dashboard	Required visibility
Executive	Project stage, centre coverage, pending confirmations, documents, audits, allocation, readiness, incidents, invoices, payments outstanding and closure readiness.
Project Manager	Project milestones, assigned team, centre pipeline, blockers, overdue work, readiness and financial summary.
Operations	Locks, bookings, consent, document gaps, calling tasks, allocation validation, readiness and incidents.
Audit / IT	Audit queue, validity, findings, corrective actions, scan status and technical exceptions.
Accounts / Finance	Quotation, approval, agreements, invoices, due amounts, paid amounts, outstanding and overdue payments.
Centre User	Own bookings, consent actions, document checklist, tasks, audit support, readiness requirements and invoice status.

- Dashboards shall read transactional data or approved reporting read models, not manually prepared reports.
- Every dashboard query shall enforce data scope and financial-data restrictions.
- Drill-down shall preserve filter context and route users to authorized record details.
- Report generation may be asynchronous for large exports.
- Exports shall record user, timestamp, filters, export type and result.

13. Security Requirements

- Implement role-based access control and client/project/centre/financial scope authorization.
- Require MFA for Admin, Project Manager, Accounts / Finance and other privileged roles.

- Use an identity provider or secure application authentication with short-lived access tokens and refresh/session controls.
- Encrypt all traffic with TLS and encrypt stored data using platform-managed encryption.
- Store secrets only in a secret manager; never commit secrets to source code or CI configuration.
- Use private object storage and short-lived signed URLs after authorization for document access.
- Validate uploaded files by extension, MIME type, size, content rules and malware scanning.
- Prevent insecure direct object references by validating scope on every object read/write.
- Apply rate limiting, security headers, input validation, dependency patching and centralized error handling.
- Maintain append-only audit logs for critical actions, approvals, overrides, sensitive reads and exports.
- Define backup, restore, retention, disaster recovery and security incident-response procedures.

14. Non-Functional Requirements

Category	Requirement
Availability	Target 99.5% monthly production availability, excluding approved maintenance.
Performance	Standard API operations should achieve p95 response under 500 ms under normal expected load; heavy imports and exports shall be asynchronous.
Dashboard performance	Standard dashboard views shall load within 5 seconds for supported volumes.
Concurrency	Conflicting bookings must not be committed when concurrent requests occur.
Scalability	Architecture shall support expected project, centre, document, device and concurrent-user volumes; load testing must validate assumptions.
Auditability	Critical transactions must record actor, action, timestamp, old value, new value, reason and approver where applicable.
Accessibility	Core workflows shall support keyboard operation and responsive behavior in supported browsers.
Maintainability	Code shall use TypeScript, modular boundaries, code review, automated tests and documented APIs.
Data retention	Retention periods shall be configurable/approved for operational, financial, document and audit records.
Observability	System shall provide centralized logs, metrics, traces, error reporting, queue health and alerting.

15. Testing Requirements

15.1 Automated Testing

Test type	Tools	Minimum coverage focus
Unit	Vitest/Jest	Validators, calculations, workflow transitions, RBAC policies and service methods.
API/integration	Supertest, test database/containers	Authorization, transactions, indexes, workflow APIs and audit logging.
Frontend	React Testing Library	Forms, validation, conditional fields, state display and permission-driven UI.
End-to-end	Playwright	Role journeys across browser, API and workflow states.
Performance	k6 or equivalent	Centre search, booking contention, allocation imports, dashboards and scan ingestion.
Security	OWASP ZAP plus review	IDOR, authorization bypass, injection, unsafe upload and rate-limit tests.
Automation	n8n test instance	Webhook processing, retries, idempotency, reminder schedules and delivery logs.

15.2 Mandatory Regression Scenarios

- Two users attempt a conflicting booking and only one can complete it.
- Cancelled booking history remains and replacement booking is linked.
- Mandatory documents block the configured next workflow stage until approved.
- Document rework creates a new version without deleting the old version.
- Auditor cannot approve the audit submitted by the same auditor.
- Duplicate MAC address creates a technical exception.
- Locked allocation cannot be directly edited and must follow controlled change request.
- Commercial revision creates a new version and requires re-approval.
- Operations cannot modify restricted payment/financial records.
- Overdue task, incident or ticket triggers reminder and escalation.
- Centre User cannot access another centre's data.
- Every critical action produces an authorized audit trail entry.

16. CI/CD Requirements

16.1 Environments

Environment	Use
Local	Developer environment using Docker Compose or equivalent with MongoDB and Redis.
Development	Shared integration environment with automated deployments and test/synthetic data.
Staging/UAT	Production-like environment for automated regression and business acceptance.
Production	Protected live environment with approval gates, monitoring, backup and rollback controls.

16.2 Pipeline

- Pull request creation triggers linting, formatting, TypeScript checks, unit tests and dependency scans.
- Build application/container images and run API/integration tests.
- Run security/container scans before deployment.
- Automatically deploy approved branches to Development.
- Deploy release candidates to Staging/UAT after required checks pass.
- Run smoke and end-to-end tests in Staging/UAT.
- Require authorized approval before production deployment.
- Perform database-index/migration tasks using controlled, reversible release procedures.
- Perform post-deployment health checks and activate rollback if checks fail.
- Publish release notes, known issues and rollback instructions for each production release.

17. DevOps and Operations Requirements

- Containerize frontend, backend, worker and n8n components using Docker.
- Use managed MongoDB and managed Redis where feasible.
- Use private container registry and pinned image versions.
- Use Infrastructure as Code, such as Terraform, for repeatable environments where practical.
- Monitor application errors, latency, database health, Redis queue depth, job failures, n8n failures and storage access.
- Create alerts for failed deployments, elevated errors, queue backlog, failed notifications, failed backups and system unavailability.

- Back up database and required configuration; perform restore tests on an approved schedule.
- Maintain runbooks for deployment, rollback, failed integrations, document-storage incidents and security incidents.

18. Delivery Plan

Phase	Scope	Exit criteria
Phase 0	SRS review, workflow clarification, wireframes, architecture, data model and test plan	Business and technical decisions approved.
Phase 1	Platform foundation: identity, RBAC, scopes, audit framework, file foundation, CI/CD	Secure users, roles and environments operational.
Phase 2	Core MVP: project, centre, eligibility, booking, consent, documents and core commercial workflow	Controlled booking journey passes automated and UAT tests.
Phase 3	Audit, scan, allocation, readiness, tasks, incidents and tickets	Operational control and exception journeys integrated.
Phase 4	Invoice, payment, inventory, communication and advanced dashboards	Finance and execution lifecycle traceable.
Phase 5	Integrations, hardening, pilot, UAT, go-live	Production readiness, support and sign-off completed.

19. Required Team Capabilities

Capability	Reason
Full-stack MERN developer	Build React UI, Node API, MongoDB integration and core workflows.
Business Analyst / Product Owner	Resolve business-rule ambiguity, prioritize backlog and coordinate UAT.
QA Automation Engineer	Create regression suite for workflows, permissions, concurrency and integrations.
UI/UX Designer	Design efficient dashboards, forms and centre-facing flows.
DevOps / Cloud Engineer	Implement CI/CD, cloud infrastructure, secrets, monitoring, backups and releases.
Security Reviewer	Review authorization, document access, financial segregation and audit controls.
Finance Process Owner	Approve commercial, invoice, payment, tax and reconciliation workflow rules.
Technical Scan Specialist	Define scan agent, network collection, payload and technical validation for M09.

20. Assumptions and Open Decisions

- Approved cloud provider, region and hosting model.
- Identity provider and MFA approach.
- Exact booking-conflict rule for date, shift and centre capacity.
- Soft-lock duration, extension and release policy.
- Audit validity and re-audit rules by client, exam and risk category.
- Document checklist, expiry policy and retention policy.
- Commercial formula, tax, payment-term and invoice matching rules.
- Approved message providers for email, SMS, WhatsApp and telephony.
- Technical scan agent design, data payload, network access and security method.
- Expected centre, project, document, device, user and concurrent-load volumes.
- Migration strategy for legacy Excel, emails, documents and centre data.

21. Acceptance and Sign-Off

The solution shall be accepted when the approved functional workflows, required security controls, role and scope permissions, automated regression scenarios, integration tests, performance expectations, backup/restore checks and UAT scenarios pass in the staging/UAT environment.

Approver role	Name	Signature	Date
Business Owner / Management			
Project Manager			
Operations Representative			
Accounts / Finance Representative			
Audit / IT Representative			
Technology / System Owner			